

Post-Training k-Quantization and Agentic Tool-Calling Accuracy on Five Qwen3 and Llama Models

Akshay Kumar¹

¹ Independent Researcher

Corresponding author:

Akshay Kumar¹

Email address: akumar8@mt.iitr.ac.in

ORCID: 0009-0006-3613-538X

Article type: Research Article

Journal: PeerJ Computer Science

Post-Training k-Quantization and Agentic Tool-Calling Accuracy on Five Qwen3 and Llama Models

Akshay Kumar

Abstract

Background. Language models run on laptops after post-training quantization, and the format that ships is the llama.cpp k-quant. The accuracy cost of that compression has been measured on prose, knowledge and reasoning benchmarks, and not on the output an agent produces, which is a function call that a program then executes.

Methods. We built a five-rung k-quant ladder, Q8_0 to Q3_K_M, from one FP16 source for Qwen3-1.7B, and three-rung ladders for Qwen3-8B, Qwen3-14B, Llama-3.2-3B and Llama-3.1-8B. Every rung was scored on the Berkeley Function Calling Leaderboard’s simple_python category by deterministic syntax-tree matching ($n = 400$ or 200), with a type-coerced secondary score, and on 400 GSM8K questions with the same weight files. Rungs are compared with McNemar’s exact test on identical items, every accuracy carries a Wilson 95% interval, and two rungs were each run three times with nothing changed, to measure the instrument. The hypotheses were pre-registered before the first run.

Results. On Qwen3-1.7B tool calling is flat from Q8_0 through Q4_K_M and loses 4.75 points at Q3_K_M ($p = 0.004$), about 10 times the run-to-run range. Free-form mathematics on those files loses 39.50 points at that rung, because 213 of 400 generations reach the 4,096-token budget while still reasoning. Qwen3-8B and Qwen3-14B show no loss at any rung. Multi-turn tool calling on Qwen3-1.7B falls from 17.00% to 1.50% at Q3_K_M. Llama-3.2-3B loses 4.00 points at Q4_K_M under the strict checker ($p = 0.021$) and 2.50 under the coerced one ($p = 0.125$). Llama-3.1-8B’s strict score rises from 50.00% at Q8_0 to 71.50% at Q3_K_M because its full-precision weights write numeric arguments as quoted strings; the coerced checker, which accepts a quoted value when it parses to what the reference holds, scores the same runs at 92.00%, 90.50% and 85.00%. A fair answer extractor moves the free-form numbers by at most 0.50 points, and a logit penalty on overthinking markers changes no tool-calling verdict.

Conclusions. Four-bit k-quantization costs no detectable single-call accuracy on any of the five models once argument typing is separated from call correctness; three-bit costs Qwen3-1.7B and Llama-3.1-8B measurable accuracy, and the other three models show no detectable loss. The length-driven collapse of chain-of-thought reasoning under three-bit weights does not reach a single tool call. A strict schema score can move in either direction under quantization because it also measures a formatting habit, so a coerced score belongs beside it.

Keywords. post-training quantization; llama.cpp; k-quant; tool calling; function calling; language model agents; BFCL; GSM8K; pre-registration

1 Introduction

A laptop runs Qwen3-1.7B in its three-bit k-quant, Q3_K_M, as the model behind a small agent. The agent is asked to find the greatest common divisor of 36 and 48 and is handed one function to call, `number_theory.gcd`. The same model at eight bits, Q8_0, answers with the call `number_theory.gcd(number1=36, number2=48)`. At three bits it answers with a sentence: “The Greatest Common Divisor (GCD) of two numbers, say 36 and 48, is 24.” The program waiting for a call receives prose, and the number in the prose is wrong as well; the answer is 12. Both outputs came from one FP16 weight file. Only the number of bits used to store each weight changed. This is item `simple_python_21` of the benchmark used below, one of 30 items the three-bit weights lose against the eight-bit weights, and the question of this paper is how often that happens, at which bit-width it starts, and whether the function call or the free-form reasoning breaks first.

The compression is post-training quantization (Dettmers et al., 2022; Frantar et al., 2023; Lin et al., 2024). The full-precision file for Qwen3-1.7B is 3.8 GB; the version most people would run, Q4_K_M, is 1.3 GB, about 34% of it. The format nearly everyone uses on a laptop is the family of llama.cpp k-quants (Gerganov and contributors, 2023), because that is what Ollama and the desktop front ends serve, and a systematic review of language-model deployment on edge devices counts model compression, quantization included, as a central enabler of running these models on memory-limited hardware (Dinçer and Kilimci, 2026). Four bits is where the scaling analysis says the trade-off is best (Dettmers and Zettlemoyer, 2023; Kumar et al., 2025), and it is the default most front ends pull. Lower bit-widths are pursued for throughput as well as size, although Malekar and Zand (2025) show, for binary and ternary weights, that the throughput gain is capped by whatever part of the model stays in higher precision.

The cost of that compression has been measured with care, on tasks other than the one an agent performs. Broad evaluations cover language modelling, classification, knowledge, dialogue, long context and trustworthiness across many families (Jin et al., 2024; Li et al., 2024); Kurtic et al. (2025) score the Llama-3.1 family under FP8, INT8 and INT4 on academic benchmarks and on chat, code-generation and long-context tasks; Lee et al. (2025) score instruction-tuned models from 1B to 405B under four quantization methods on thirteen datasets; multilingual degradation has its own study (Marchisio et al., 2024); long-context inputs lose up to 59% under four-bit methods (Mekala et al., 2025); reasoning models have been quantized and scored on mathematics and code (Liu et al., 2025). Kurt (2026) quantizes Llama-3.1-8B-Instruct into thirteen GGUF formats and scores each on GSM8K, HellaSwag, IFEval, MMLU, TruthfulQA and perplexity. Lotfi et al. (2026) measure five reasoning models under GPTQ, AWQ and FlatQuant on mathematics, code and science questions and find accuracy falling as chain-of-thought inflates. Every task in every one of these papers is free-form or multiple choice. None involves emitting a function call.

An agent calls tools (Patil et al., 2024; Qin et al., 2024; Yao et al., 2025). It emits a function name and a set of typed arguments that a program will execute. That output either passes the checker or it does not, with no partial credit and no reader to accept an approximate answer. Our starting intuition was that a function signature has none of the slack prose has, so compression should break tool calling at a higher bit-width than it breaks free-form accuracy. We pre-registered this as

our first hypothesis before running anything.

We built a controlled ladder, five k-quant formats produced from one FP16 source, and scored each rung on the Berkeley Function Calling Leaderboard (Patil et al., 2025), which compares a model’s call to a reference by deterministic abstract-syntax-tree matching; no model judges another model anywhere in the pipeline. We then ran each rung’s file on a free-form control so that the two floors could be compared on identical parameters, repeated the three rungs that matter on four further models in two families, and re-scored every run under a checker that accepts a correctly valued argument in the wrong type.

For Qwen3-1.7B the answer is the reverse of the one we expected. Tool-calling accuracy is flat from Q8_0 through Q4_K_M and falls by 4.75 points at Q3_K_M, roughly 10 times the run-to-run noise of the instrument. On those files free-form mathematics falls by 39.50 points at the same rung. Both floors sit between four and three bits, but the drop in tool calling is about an eighth of the drop in mathematics. The difference comes down to output length. At three bits the model’s chain of thought (Wei et al., 2022) grows until 53% of math generations use up their token budget before reaching an answer. A tool call is a median of 182 tokens, so the weights never get the room to wander. On Qwen3-8B and Qwen3-14B there is no cliff at any rung. On the Llama family the picture is different, and part of the difference is the checker rather than the weights: Llama-3.2-3B loses at four bits under strict scoring and not under coerced scoring, and Llama-3.1-8B’s strict score rises under compression because its full-precision weights quote numbers, a habit the compressed weights partly lose (§4.5).

Contributions. Each item names the table or figure that holds it.

1. A pre-registered, controlled k-quant ladder for tool calling: 5 rungs from one FP16 source on Qwen3-1.7B ($n = 400$) and 3 rungs on 4 further models ($n = 200$), scored by a deterministic checker and by a type-coerced secondary checker, with McNemar tests on identical items and Wilson 95% intervals on every accuracy (§3, Tables 2 and 4).
2. An instrument measurement most quantization papers omit: identical weights re-run 3 times move by at most 0.50 points and flip at most 4 of 400 verdicts (Table 1).
3. The Qwen3-1.7B floor: no detectable loss down to Q4_K_M, then 4.75 points at Q3_K_M, $p = 0.004$, 30 lost against 11 gained (Table 2); no loss at any rung on Qwen3-8B or Qwen3-14B (Table 4).
4. The free-form comparison on the same weight files: 39.50 points at Q3_K_M with 213 of 400 generations at the 4,096-token budget (Table 3, Figure 1); a 16,384-token re-run of those 213 items leaves 77% still at the cap.
5. Multi-turn tool calling on Qwen3-1.7B: 17.00% at Q8_0 to 1.50% at Q3_K_M, $n = 200$, $p < 0.0001$ (Table 6).
6. A second family scored two ways: Llama-3.2-3B loses 4.00 points at Q4_K_M strict ($p = 0.021$) and 2.50 coerced ($p = 0.125$); Llama-3.1-8B gains 21.50 strict points at Q3_K_M and loses 7.00 coerced points ($p = 0.003$), with 84 of its 100 Q8_0 strict failures being quoted values of the right number (Tables 4 and 5).

7. Two pre-registered nulls: a fair extractor changes free-form accuracy by at most 0.50 points at any rung (Table 3), and a logit penalty on overthinking markers changes tool-calling accuracy by 0.00 points at Q3_K_M (§4.7).

2 Related work

Quantization evaluated on free-form tasks. The broad evaluations of post-training quantization score many model families on language modelling, multiple-choice knowledge, dialogue, long-context and trustworthiness tasks (Jin et al., 2024; Li et al., 2024); a full-text search of both finds no tool-calling or function-calling benchmark. Kurtic et al. (2025) run more than half a million evaluations on the Llama-3.1 family under FP8, INT8 and INT4 and find FP8 effectively lossless, well-tuned INT8 1 to 3% below the BF16 baseline, and INT4 weight-only quantization on par with 8-bit; Lee et al. (2025) find quantized models generally ahead of smaller full-precision ones but weaker on instruction following and hallucination detection. Neither scores a function call. Liu et al. (2025) quantize the DeepSeek-R1-distilled and QwQ reasoning models and find mathematics and code degrading under aggressive weight and KV-cache quantization, again on free-form tasks. Kurt (2026) is the closest work and the one this paper extends: one model, thirteen llama.cpp formats from Q3_K_S to Q8_0 against FP16, six free-form or multiple-choice benchmarks, plus throughput and size. A full-text read finds no tool-calling, function-calling, agentic, structured-output or schema-compliance evaluation, and its conclusion asks for exactly what we add, additional model families and sizes under the same protocol. Lotfi et al. (2026) take a different axis, in the line of work on overthinking in reasoning models (Chen et al., 2024; Sui et al., 2025). Under GPTQ (Frantar et al., 2023), AWQ (Lin et al., 2024) and FlatQuant, five reasoning models lose accuracy while emitting longer chains of thought, and in up to 52% of failures the correct answer appears mid-trace without being committed to. Their compression methods are research methods rather than the k-quants that ship to laptops, and their failure categorisation, including the overthinking class, relies on GPT-5 as a judge. Setting quantization aside, the gain from zero-shot chain-of-thought prompting over direct answering holds on average across six models and six question-answering datasets but varies in size by model and dataset (Hebenstreit et al., 2024); what quantization does to the length of the chain is a further axis. Surveys of language models for constrained deployment list quantization among the standard compression techniques (Turkmen, 2026; Dinçer and Kilimci, 2026), and the survey of small clinical models names agentic tool use as a future direction (Turkmen, 2026); neither measures the two together.

Tool-calling evaluation. Gorilla (Patil et al., 2024) and ToolLLM (Qin et al., 2024) established API calling as a benchmark task, and τ -bench (Yao et al., 2025) extends it to multi-turn agent-user interaction. BFCL (Patil et al., 2025) scores a model’s function call against a reference by abstract-syntax-tree matching over the function name, argument names, types and values, across single, multiple and parallel calls, an irrelevance category where no call is correct, and multi-turn trajectories. Because the matcher is deterministic, the same generation always receives the same verdict, so a difference between two rungs can be attributed to the weights rather than to the scorer. Dondeti (2026) study the same class of small models on consumer hardware from the safety side, scoring adversarial tool calls from seven models between 1B and 4B parameters with

a deterministic would-dispatch rule; this paper is the accuracy side of the same deployment.

The gap. Quantization has been scored on prose, knowledge, instruction following and reasoning, for research compression methods and for the deployed one. It has not been scored on tool calling under either. That is what we measure.

3 Materials and Methods

3.1 Pre-registration

Hypotheses and thresholds were fixed on 18 September 2026 before the first grid run and are in paper/preregistration.md in the repository. Two full-text summaries of prior work were appended to that file during the first rung as documentation; no hypothesis or threshold changed after runs began.

3.2 Problem statement

A *ladder* for a model M is an ordered list of weight files $r_0, r_1, \dots, r_K$ produced from one FP16 source with one quantizer, in decreasing bits per weight, so that the files differ in their weights and in nothing else. Here r_0 is always Q8_0. An *item set* I is a fixed list of n benchmark items with reference answers. A *verdict* $v(i, r) \in \{0, 1\}$ is the deterministic output of a checker on the model’s response to item i served from rung r , and *accuracy* $A(r)$ is the mean verdict over I . Two verdict functions are used for tool calling. The *strict* verdict is BFCL’s abstract-syntax-tree match. The *coerced* verdict equals the strict verdict except that a strict failure is re-checked by the same checker after every string-typed argument whose schema declares a non-string type is replaced by its parsed value when it parses as that type, so a quoted number, list or boolean counts when the parsed value is what the reference holds and not otherwise. The coerced verdict is never below the strict one.

Rungs are compared to r_0 pairwise on the same items. The test is McNemar’s exact test on the discordant pairs (McNemar, 1947): *lost* counts items with $v(i, r_0) = 1$ and $v(i, r) = 0$, *gained* the reverse, and p is the two-sided exact binomial probability of a split at least that uneven under the null of no difference. Every accuracy carries a Wilson score 95% interval, which is reported in square brackets. The *cliff rung* $c(M)$ is the highest-precision rung r_k , $k \geq 1$, with $A(r_k) < A(r_0)$ and $p < 0.05$ against r_0 . The *floor* is the rung immediately above the cliff rung, the lowest-bit rung on the ladder with no detected loss. A model is *flat* on the ladder when it has no cliff rung, so its floor is at or below the lowest rung measured. A detected loss must also exceed the *instrument tolerance* τ , the largest range in accuracy across three runs of identical weights and settings (§4.1); on this instrument $\tau = 0.50$ points, and every loss reported as a cliff below is at least 10 times larger. Item counts are $n = 400$ for the Qwen3-1.7B ladder and both free-form arms, and $n = 200$ for the other four models and for the multi-turn category. Every rung is a single run except the two Qwen3-1.7B rungs used for the tolerance measurement, so a stranger who repeats one rung should expect its accuracy to move within τ and its verdicts on a few items to flip.

Under these definitions the cliff rung is Q3_K_M for Qwen3-1.7B under both verdicts, none for Qwen3-8B and none for Qwen3-14B (both flat), Q4_K_M strict and none coerced for Llama-3.2-3B, and none strict and Q3_K_M coerced for Llama-3.1-8B. The Results section gives the numbers behind each.

3.3 The ladder

Every rung of Qwen3-1.7B (Qwen Team, 2025) is produced from a single FP16 GGUF with llama-quantize. The formats are a subset of the ladder in Kurt (2026), chosen to span the range at one rung per bit-width: Q8_0, Q6_K, Q5_K_M, Q4_K_M and Q3_K_M. Ollama’s library publishes only two of these five for this model, Q8_0 and Q4_K_M, alongside FP16, which is why the ladder is built rather than pulled. A Qwen3-4B rung was planned and dropped: bfc1-eval has no entry for the original Qwen3-4B, only for the later Instruct-2507 lineage, and mixing lineages would have confounded the size comparison. Four further models get the three rungs that turned out to matter, Q8_0, Q4_K_M and Q3_K_M, each from its own FP16 source with the same quantizer: Qwen3-8B and Qwen3-14B from the same family, and Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct from a second family, whose Llama 3.1 models are described in Llama Team, AI @ Meta (2024). The FP16 sources are the *-fp16 blobs Ollama distributes. Per model the design is: Qwen3-1.7B, 5 rungs, $n = 400$, with 3 runs at Q8_0 and Q4_K_M; Qwen3-8B, Qwen3-14B, Llama-3.2-3B and Llama-3.1-8B, 3 rungs each, $n = 200$, 1 run per rung. Both Llama models are instruction-tuned rather than reasoning-tuned, which is deliberate: it separates what the ladder does to a tool call from what it does to a chain of thought. Llama-3.1-8B-Instruct is also the model Kurt (2026) quantized, so the two ladders meet on one model.

3.4 Serving

Each rung is served by llama-server from llama.cpp 0.4.1 (Homebrew; build 10964, commit b29c606e2 for the binary in use at the time of writing) with full Metal offload on an Apple M1 Max with 64 GB of memory, under a fixed model alias, so the harness sends identical requests to every rung. The single-call ladder and the repeats use four slots and an 8,192-token context; the free-form control, the larger models, the Llama models and the multi-turn runs use 32,768, because reasoning traces overflowed the smaller context; the 16,384-token budget re-run uses 65,536 with two slots. Generation is at temperature 0.001, the bfc1-eval default, for both arms. No sampling seed is fixed: llama-server is run with its default seed of -1 , which draws a fresh random seed per request, so decoding is near-greedy but not bit-identical between runs. The repeat control below measures what that leaves. For the tool-calling arm, bfc1-eval 2026.3.23 builds the prompt itself in each model’s own function-calling format and sends it to the server’s raw completions endpoint, so the server’s --jinja chat template is not involved in those runs; the free-form arm uses the chat completions endpoint, where the template applies. Measured throughput is about 35 tokens per second per slot and 650 tokens per second of prompt evaluation. Python is 3.12.

3.5 Scoring

BFCL v4 (Patil et al., 2025), `simple_python` category, deterministic AST matching, run through the `-FC` handler of each model so that the model uses its native function-calling format. The strict verdict is the checker’s own. The coerced verdict is computed afterwards from the score files, with no new generation: each strict failure is passed back through BFCL’s `simple_function_checker` with string arguments parsed to the type the function schema declares (`analysis/coerced.py` in the repository). A quoted integer is parsed only if it is an integer literal, a quoted list only if it parses as a list, a quoted boolean only if it is `true` or `false`; anything else is left as the string it was, and the checker then compares the parsed value with the reference exactly as it would a natively typed one, including nested types and optional parameters. The Qwen3 runs are unchanged by this re-score: 0 of their 226 strict failures across all rungs are of the coercible kind. Differences between rungs are tested with McNemar’s exact test on the discordant pairs and every accuracy carries a Wilson 95% interval, as defined in §3.2.

3.6 Reproducibility control

Two rungs of Qwen3-1.7B, Q8_0 and Q4_K_M, were run three times each with nothing changed between runs. This gives the run-to-run variation of the instrument, which is needed before any difference between rungs can be read as a difference between weights. The control was run on the 1.7B model only; the other four models are single runs and inherit its tolerance as an assumption, which the Limitations section returns to.

3.7 Free-form control

The same five Qwen3-1.7B weight files, and the three Llama files of each Llama model, answer 400 GSM8K test questions (Cobbe et al., 2021), drawn at seed 42, through the same server under the same alias, with a 4,096-token completion budget. Every generation is logged in full so that scoring under a different extractor is a re-read rather than a re-run. Two extractors were fixed before any rung was scored. Strict accepts an explicit commitment in any of three forms: “The answer is *N*”, “#### *N*”, or `\boxed{N}`. Lenient takes the last number in the visible answer. We record that our first strict rule accepted only the first form and would have scored 21 of the first 59 generations wrong for writing the answer in a box; that rule was a strawman, it was replaced before scoring, and the gap it would have produced is not reported anywhere in this paper.

3.8 Check against the published leaderboard

Llama-3.1-8B-Instruct at Q8_0 scores 50.00% under the strict checker here, which is far below what the public leaderboard reports for that model, so the comparison is stated before the results. The BFCL leaderboard data file (Gorilla Team, UC Berkeley, 2026), last modified 13 April 2026 and retrieved on 2026-09-29, lists Llama-3.1-8B-Instruct in prompting mode only, as “Llama-3.1-8B-Instruct (Prompt)”, at 94.00% on Python simple AST over the full 400 items; it

lists no function-calling row for the model (0 of its 1 Llama-3.1-8B rows). Our run uses the function-calling handler, in which `bfc1-eval` builds Meta’s JSON tool-call prompt for Llama 3.1 itself and sends it to the raw completions endpoint of `llama-server`, with the tokenizer and configuration loaded from an ungated mirror of the same files because the `meta-llama` repositories are gated (`REMOTE_OPENAI_BASE_URL` and `REMOTE_OPENAI_TOKENIZER_PATH` in `run_llama.sh`). In that format the full-precision model writes numeric arguments as quoted strings, which the strict checker rejects. The coerced score at Q8_0, 92.00% [87.4, 95.0] on the first 200 items, is consistent with the published prompting-mode number, so the gap between 50.00% and 94.00% is the argument-typing habit under the JSON tool-call format, not the weights or the server. A cross-check of the same GGUF files on a second inference server has not been run.

3.9 Overthinking penalty

Lotfi et al. (2026) report that a fixed logit penalty on a curated list of overthinking markers cuts chain-of-thought length by 12 to 23% while preserving accuracy. We apply a fixed penalty of -2.0 to the eight Qwen3 tokens for *wait*, *but*, *alternatively* and *hmm* in both capitalisations, at the server, and re-run the Qwen3-1.7B ladder at Q4_K_M and Q3_K_M. This approximates their method with a single fixed λ rather than their tuned value.

3.10 Reproducibility

Every number in this paper is a macro recomputed from the raw BFCL result and score files and the logged generations; nothing is typed by hand. The scripts, all in the repository named in the Data Availability statement, are: `analysis/paper_numbers.py`, which writes `paper/tables/numbers.tex` and every table from `runs/`; `analysis/coerced.py`, the coerced re-score; `run_ladder.sh`, which rebuilds the Qwen3-1.7B ladder from the FP16 source and scores it; `run_model.sh`, which does the same for Qwen3-8B, Qwen3-14B and the multi-turn category; `run_llama.sh` and `run_gsm8k_model.sh`, which build and score the two Llama models on both arms; and `run_repeats.sh`, `run_gsm8k.sh`, `run_budget.sh` and `run_h4.sh` for the reproducibility, free-form, budget and penalty arms. The GGUF weight files are not in the archive; `run_ladder.sh` and `run_model.sh` rebuild them from the published FP16 checkpoints.

4 Results

4.1 The instrument is reproducible

Table 1 gives three runs of two rungs with nothing changed between them. Q8_0 moved by 0.25 points across three runs and Q4_K_M by 0.50. At the level of individual questions, at most 1 of 400 verdicts changed between two Q8_0 runs and at most 4 between two Q4_K_M runs. The instrument is close to deterministic, so any difference between rungs larger than about 0.50 points can be attributed to the weights rather than to run-to-run variation.

Rung	Accuracy per run	Range (pp)	Max per-question flips
Q8_0	91.75, 91.50, 91.50	0.25	1 of 400
Q4_K_M	92.00, 91.50, 92.00	0.50	4 of 400

Table 1: Run-to-run variation with nothing changed between runs, Qwen3-1.7B, `simple_python`, $n = 400$.

Rung	Size (GB)	Accuracy [95% CI]	vs. Q8_0	Lost	Gained	McNemar p
Q8_0	2.2	91.75% [88.6, 94.1]	baseline			
Q6_K	1.7	93.50% [90.6, 95.5]	+1.75 pp	6	13	0.167
Q5_K_M	1.5	90.75% [87.5, 93.2]	-1.00 pp	10	6	0.454
Q4_K_M	1.3	92.00% [88.9, 94.3]	+0.25 pp	13	14	1.000
Q3_K_M	1.1	87.00% [83.3, 89.9]	-4.75 pp	30	11	0.004

Table 2: The quantization ladder for Qwen3-1.7B on `simple_python`, $n = 400$ per rung, all rungs from one FP16 source, one run per rung. Brackets are Wilson 95% intervals. Lost and gained count questions that flip against Q8_0; p is McNemar’s exact test on those discordant pairs, in bold where below 0.05.

4.2 Flat to four bits, a cliff at three

Table 2 is the ladder. From Q8_0 down to Q4_K_M the accuracy sits in a band of under three points and no rung differs from Q8_0 at any conventional level: Q4_K_M is at 92.00% [88.9, 94.3] against Q8_0’s 91.75% [88.6, 94.1], a difference of +0.25 points with a McNemar p of 1.000, while being 59% of the file size. Q6_K scores above Q8_0 and Q5_K_M below it, and neither difference survives paired testing, so the apparent non-monotonicity in the raw numbers is within the instrument’s noise.

Q3_K_M is a different case. It loses 30 questions that Q8_0 gets right and recovers 11, a net loss of 4.75 points at $p = 0.004$. That drop is about 10 times the largest run-to-run range in Table 1, which places the tool-calling floor for this model at Q4_K_M and the cliff rung at Q3_K_M. Most of the 30 lost items are calls with a wrong argument value; the rest are prose in place of a call, as in the GCD item of the introduction, or a wrong function name.

4.3 The free-form floor on the same weights

Figure 1 puts both tasks on one axis and Table 3 gives the numbers. The tool-calling line barely moves while the free-form line drops sharply at the same rung, and the right panel shows the reason. The free-form floor sits exactly where the tool-calling floor sits, at Q4_K_M, and nowhere else: no rung above Q3_K_M differs from Q8_0 under paired testing. So H1 as pre-registered fails, and it fails in the opposite direction. At Q3_K_M free-form accuracy falls by 39.50 points, against 4.75 for tool calling. The output with no slack is the one that holds up.

The mechanism is in the last two columns of the table. At Q3_K_M, 213 of 400 generations hit the 4,096-token completion budget while still inside the chain of thought and never emit an

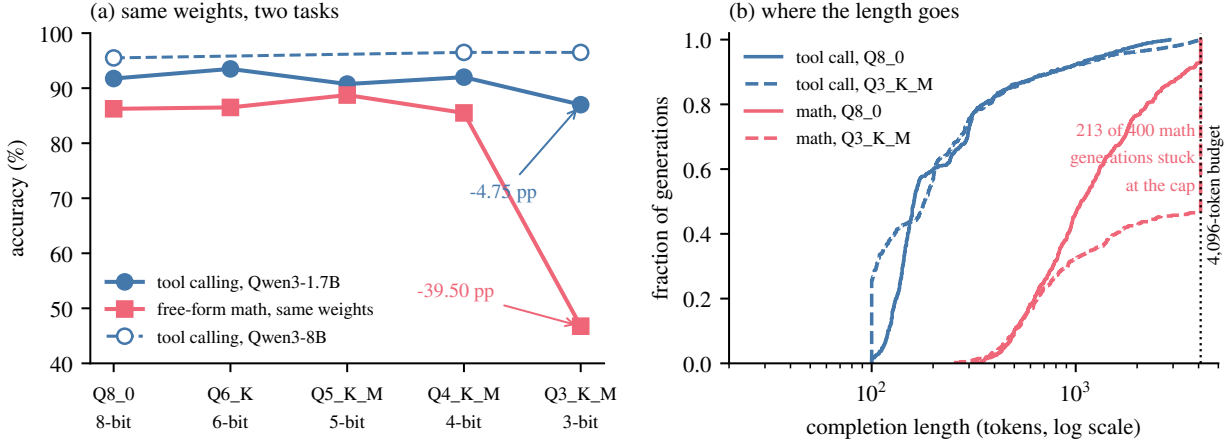


Figure 1: (a) Accuracy across the quantization ladder on identical weight files: tool calling on BFCL and free-form mathematics on GSM8K for Qwen3-1.7B, with Qwen3-8B tool calling dashed. Both floors sit at Q4_K_M; the free-form drop at Q3_K_M is about eight times the tool-calling drop. (b) Cumulative distribution of completion length at Q8_0 and Q3_K_M for each task. Tool calls are short at both rungs, while math generations at Q3_K_M pile up against the 4,096-token budget and 213 of 400 never finish.

Rung	Strict [95% CI]	Lenient	Extractor gap	vs. Q8_0 (strict)	McNemar p	Median tokens
Q8_0	86.25% [82.5, 89.3]	86.50%	+0.25 pp	baseline		1065
Q6_K	86.50% [82.8, 89.5]	86.50%	+0.00 pp	+0.25 pp	1.000	1102
Q5_K_M	88.75% [85.3, 91.5]	89.25%	+0.50 pp	+2.50 pp	0.164	955
Q4_K_M	85.50% [81.7, 88.6]	85.75%	+0.25 pp	-0.75 pp	0.736	1048
Q3_K_M	46.75% [41.9, 51.6]	46.75%	+0.00 pp	-39.50 pp	<0.0001	4096

Table 3: The free-form control, GSM8K, Qwen3-1.7B, $n = 400$ per rung, identical weight files to Table 2. Brackets are Wilson 95% intervals on the strict score. Extractor gap is lenient minus strict. Median tokens is the completion length; the budget was 4,096.

answer. The median completion length at this rung is the budget itself. This is the behaviour Lotfi et al. (2026) report under GPTQ and AWQ, reproduced here on the deployed format with a deterministic scorer. Among the 187 generations that did finish, accuracy is 84.5%, against 92.7% for finished generations at Q8_0, so a smaller reasoning loss of about 8.2 points sits underneath the collapse. A tool call does not pay that cost. At Q3_K_M the median tool-call generation is 182 tokens, the 90th percentile 753, and 1 of 539 requests came near the budget. The generation ends before the same instability has room to develop.

The extractor. The gap between strict and lenient scoring is at most 0.50 points at any rung. H3 predicted a gap of at least five points that would bound how much published free-form degradation is the parser. Once the strict extractor accepts the commitment format that reasoning models actually emit, no such gap exists. The failures at Q3_K_M are generations that contain no answer at all rather than answers a parser failed to find.

The budget. A collapse driven by truncation could be an artefact of the 4,096-token cap rather

Model	Rung	n	Strict [95% CI]	vs. Q8_0	p	Coerced [95% CI]	vs. Q8_0	p
Qwen3-1.7B	Q8_0	400	91.75% [88.6, 94.1]	baseline		91.75% [88.6, 94.1]	baseline	
Qwen3-1.7B	Q4_K_M	400	92.00% [88.9, 94.3]	+0.25 pp	1.000	92.00% [88.9, 94.3]	+0.25 pp	1.000
Qwen3-1.7B	Q3_K_M	400	87.00% [83.3, 89.9]	−4.75 pp	0.004	87.00% [83.3, 89.9]	−4.75 pp	0.004
Qwen3-8B	Q8_0	200	95.50% [91.7, 97.6]	baseline		95.50% [91.7, 97.6]	baseline	
Qwen3-8B	Q4_K_M	200	96.50% [93.0, 98.3]	+1.00 pp	0.625	96.50% [93.0, 98.3]	+1.00 pp	0.625
Qwen3-8B	Q3_K_M	200	96.50% [93.0, 98.3]	+1.00 pp	0.688	96.50% [93.0, 98.3]	+1.00 pp	0.688
Qwen3-14B	Q8_0	200	96.00% [92.3, 98.0]	baseline		96.00% [92.3, 98.0]	baseline	
Qwen3-14B	Q4_K_M	200	96.00% [92.3, 98.0]	+0.00 pp	1.000	96.00% [92.3, 98.0]	+0.00 pp	1.000
Qwen3-14B	Q3_K_M	200	96.50% [93.0, 98.3]	+0.50 pp	1.000	96.50% [93.0, 98.3]	+0.50 pp	1.000
Llama-3.2-3B	Q8_0	200	92.00% [87.4, 95.0]	baseline		93.50% [89.2, 96.2]	baseline	
Llama-3.2-3B	Q4_K_M	200	88.00% [82.8, 91.8]	−4.00 pp	0.021	91.00% [86.2, 94.2]	−2.50 pp	0.125
Llama-3.2-3B	Q3_K_M	200	86.50% [81.1, 90.6]	−5.50 pp	0.007	91.00% [86.2, 94.2]	−2.50 pp	0.180
Llama-3.1-8B	Q8_0	200	50.00% [43.1, 56.9]	baseline		92.00% [87.4, 95.0]	baseline	
Llama-3.1-8B	Q4_K_M	200	63.00% [56.1, 69.4]	+13.00 pp	< 0.0001	90.50% [85.6, 93.8]	−1.50 pp	0.453
Llama-3.1-8B	Q3_K_M	200	71.50% [64.9, 77.3]	+21.50 pp	< 0.0001	85.00% [79.4, 89.3]	−7.00 pp	0.003

Table 4: Tool calling across five models on `simple_python`, under the strict BFCL verdict and the type-coerced verdict of §3.2. Qwen3-1.7B at $n = 400$; the other four at $n = 200$, one run per rung. Brackets are Wilson 95% intervals. Each model is compared against its own Q8_0 by McNemar’s exact test; p in bold where below 0.05.

than of the weights. We re-ran the 213 Q3_K_M generations that hit the cap with a budget of 16,384 tokens, four times the original. Of the 196 that completed without a server error, 151 (77%) hit the larger cap as well, and 48 (24%) reached a correct answer. Given four times the room, the three-bit model mostly keeps thinking. A larger budget converts most of the truncations into longer truncations.

4.4 Across models, strict and coerced

Table 4 gives every model at the three rungs that matter, under both verdicts. On Qwen3-8B there is no cliff. Q3_K_M scores 96.50% [93.0, 98.3] against 95.50% [91.7, 97.6] at Q8_0, a difference of +1.00 points at $p = 0.688$, and Q4_K_M is likewise indistinguishable from the baseline. Qwen3-14B repeats the pattern: 96.00% at Q8_0, 96.00% at Q4_K_M and 96.50% at Q3_K_M, with the Q3_K_M difference at +0.50 points and $p = 1.000$. On this family the three-bit cliff that binds at 1.7B has moved below the ladder by 8B and stays there at 14B, which is the direction the free-form literature reports for larger models and is now shown for the call itself at two sizes. The coerced columns for the three Qwen3 models are identical to the strict ones, because none of their strict failures is a quoted value of the right type. Both Llama models are single runs at $n = 200$, and their intervals are correspondingly wider than the 1.7B ones.

4.5 A second family

The Qwen3 pattern does not transfer to the Llama models, and the two verdicts disagree about how much of what happens is capability. Under the strict checker Llama-3.2-3B loses 4.00 points at Q4_K_M ($p = 0.021$, 9 lost against 1 gained) and 5.50 at Q3_K_M ($p = 0.007$), so under strict

Model	Rung	Strict failures	Type errors	Decode errors	Coercible	GSM8K strict [95% CI]	vs. Q8_0	<i>p</i>
Llama-3.2-3B	Q8_0	16	6	0	3	84.00% [80.1, 87.3]	baseline	
Llama-3.2-3B	Q4_K_M	24	10	0	6	81.25% [77.1, 84.8]	−2.75 pp	0.126
Llama-3.2-3B	Q3_K_M	27	13	0	9	75.50% [71.1, 79.5]	−8.50 pp	<0.0001
Llama-3.1-8B	Q8_0	100	90	3	84	91.00% [87.8, 93.4]	baseline	
Llama-3.1-8B	Q4_K_M	74	61	6	55	87.75% [84.2, 90.6]	−3.25 pp	0.041
Llama-3.1-8B	Q3_K_M	57	32	14	27	83.25% [79.3, 86.6]	−7.75 pp	<0.0001

Table 5: The second family. Failure types on `simple_python` ($n = 200$ per rung, one run): strict failures, of which type errors and decode failures, and the number of strict failures that pass the coerced checker. GSM8K strict accuracy, $n = 400$, with Wilson 95% intervals, each model against its own Q8_0 by McNemar’s exact test.

scoring its cliff rung is Q4_K_M. Under the coerced checker the same runs lose 2.50 and 2.50 points, at $p = 0.125$ and $p = 0.180$, and the model has no cliff rung on the ladder. The difference is in Table 5: of its strict failures, 3 at Q8_0, 6 at Q4_K_M and 9 at Q3_K_M are the right value in a quoted string, so the strict loss overstates the loss in call correctness: 4.00 against 2.50 points at Q4_K_M and 5.50 against 2.50 at Q3_K_M, the remainder being a change in how the compressed weights type their arguments.

Llama-3.1-8B is the extreme case. Its Q8_0 weights score 50.00% under the strict checker; Q4_K_M and Q3_K_M score 63.00% and 71.50%, both at $p < 0.0001$ against Q8_0, an inversion of the expected order. Table 5 gives the reason. Of the 100 Q8_0 strict failures, 90 are type errors, and 84 of those pass the checker once the quoted value is parsed: the model names the right function and the right value and writes the number in quotes, as in `{"base": "10", "height": "5"}` for a triangle with base 10 and height 5. At Q3_K_M only 27 of 57 failures are of that kind, while decode failures, where the output does not parse as a call at all, rise from 3 to 14. The coerced column of Table 4 restores the expected order: 92.00% [87.4, 95.0], 90.50% [85.6, 93.8] and 85.00% [79.4, 89.3] down the ladder, no detectable loss at Q4_K_M ($p = 0.453$) and a 7.00-point loss at Q3_K_M ($p = 0.003$). So the compressed 8B weights write bare numbers more often, which raises the strict score, while the underlying call holds at four bits and gets worse at three. Free-form accuracy on the same files falls 3.25 points at Q4_K_M and 7.75 at Q3_K_M. Length plays no part in either Llama model: GSM8K completions have a median of 185 to 220 tokens, and at most 5 of 400 reach the budget at any rung, so the run-away behind the Qwen3 collapse cannot occur and the free-form losses here are ordinary precision loss.

What the second family adds is one finding and one warning. The finding is that quantization changes argument typing before it changes call correctness: on both Llama models the count of coercible failures moves with the rung while the coerced accuracy at Q4_K_M does not differ from Q8_0. The warning is that a strict schema score can move in either direction under quantization, because it measures a formatting habit as much as a capability, and a reader given only the strict column of Table 4 would conclude that compressing Llama-3.1-8B improves it. Anyone scoring tool calls under compression should report the failure types and a coerced score beside the strict one.

Rung	<i>n</i>	Accuracy [95% CI]	vs. Q8_0	Lost/Gained	McNemar <i>p</i>	Relative (%)	Single call, relative (%)
Q8_0	200	17.00% [12.4, 22.8]	baseline				
Q4_K_M	200	13.50% [9.4, 18.9]	-3.50 pp	16/9	0.230	-20.6	+0.3
Q3_K_M	200	1.50% [0.5, 4.3]	-15.50 pp	32/1	<0.0001	-91.2	-5.2

Table 6: Multi-turn tool calling, BFCL `multi_turn_base`, Qwen3-1.7B, $n = 200$ per rung, one run. Brackets are Wilson 95% intervals. Relative is the change against Q8_0 as a fraction of the Q8_0 accuracy; the last column is the same quantity for single calls at that rung (Table 2).

4.6 Multi-turn trajectories

The pre-registration predicted that multi-turn trajectories, where an error in one call corrupts the state for every call after it, would degrade at least twice as fast as single calls. They degrade far faster than that. Table 6 gives BFCL’s `multi_turn_base` on Qwen3-1.7B, $n = 200$: 17.00% at Q8_0, 13.50% at Q4_K_M and 1.50% at Q3_K_M. The Q4_K_M drop is -20.6% relative against +0.3% for single calls at the same rung, in the predicted direction and not significant at this n ($p = 0.230$). The Q3_K_M drop is -91.2% relative against -5.2% for single calls, at $p < 0.0001$, with 32 trajectories lost and 1 gained. H5 is supported at three bits by a wide margin.

The size of the multi-turn collapse follows from §4.3. A single call is a short generation and survives three-bit weights because it ends before the length instability has room to develop. A multi-turn trajectory is many generations in sequence, each conditioned on the accumulated state, so the same per-step failure rate compounds and a single malformed call anywhere in the sequence fails the whole trajectory. The server logs show the instability reaching the call once calls are chained: at Q3_K_M the median generation per request is 509 tokens against 213 at Q8_0, and the 90th percentile 2403 against 773. Weights that keep a single call to 182 tokens let a conditioned call run more than twice as long, and the extra length is the run-away from §4.3, now with a schema to break. The low absolute base rate, 17.00% even at Q8_0, is a property of the category for a 1.7B model and is why the Q4_K_M comparison is underpowered; it does not affect the Q3_K_M conclusion.

4.7 Does the overthinking penalty transfer

The penalty has no measurable effect. With it, Qwen3-1.7B scores 92.00% at Q4_K_M and 87.00% at Q3_K_M, identical to the unpenalised runs to two decimals; at each rung 2 questions flip one way and 2 the other. H4 predicted a loss of at least three points and there is no loss. The explanation follows from §4.3: the penalty acts on tokens a reasoning trace emits and a tool call does not, so on a 182-token output there is almost nothing for it to act on. This agrees with the authors’ own scope statement that the marker list was curated for free-form English reasoning. Both the failure they describe and the fix for it live in the reasoning trace, and the tool call contains very little of that trace.

5 Discussion

Three of the five pre-registered hypotheses did not hold, and the paper reports them as they fell. H1, that the schema would break first, reversed. H3, that the answer extractor would account for at least five points of published free-form degradation, found a gap of at most 0.50 points once the extractor accepted the format reasoning models emit. H4, that the overthinking penalty would cost schema compliance, found a change of 0.00 points. H2, on the failure-mode shift, is answered partly by the Llama failure types in Table 5, where decode failures on Llama-3.1-8B rise from 3 to 14 across the ladder, and H5, on multi-turn amplification, held by a wide margin.

The tool-calling floor is a property of the model, not of the format. Four bits costs nothing detectable on any of the five models once argument typing is separated from call correctness; three bits costs Qwen3-1.7B 4.75 points and Llama-3.1-8B 7.00 coerced points, and costs Qwen3-8B, Qwen3-14B and Llama-3.2-3B nothing detectable at these sample sizes. The mechanism that damages free-form reasoning at three bits on Qwen3, a chain of thought that runs past its budget, does not reach a single tool call because the call is over long before the run-away starts, and it does reach a multi-turn trajectory because chained calls grow long enough for it to start.

On the measurement itself, two things we would tell another group. First, report the failure types. The Llama-3.1-8B inversion in the strict column of Table 4 would read as a broken harness to anyone who did not see Table 5, and it took a re-score of the existing files, not a new run, to show that 84 of 100 failures were the right number in quotes. Second, measure the instrument. Three runs with nothing changed moved by at most 0.50 points (Table 1); without that figure, none of the other differences in this paper could be attributed to anything.

Practical reading. For anyone deploying an agent on a laptop: four bits is safe for single calls on every model measured here, three bits is not on Qwen3-1.7B or Llama-3.1-8B, the bit-width should be measured on the model being shipped rather than assumed from another family, and a strict schema score should be read together with a coerced one. A model that emits malformed or wrong tool calls can take real actions, so the multi-turn collapse at three bits in §4.6 is a reason to test agents at the exact quantization they ship with.

Limitations. Three sizes of one model family and two sizes of a second, on the easiest BFCL category plus one multi-turn set for Qwen3-1.7B only, on one hardware platform and one inference server. The ladder is one quantization family; GPTQ, AWQ and the other research methods are deliberately out of scope, and multilingual degradation is excluded because Marchisio et al. (2024) already cover it. The reproducibility control was run on two rungs of one model rather than on every rung of every model, so the tolerance of 0.50 points is measured on Qwen3-1.7B and assumed for the other four; the Llama-3.2-3B strict result at Q4_K_M rests on 9 lost against 1 gained from a single run, and a repeat on that model is the first thing we would add. Four of the five models were scored on the first 200 items of the category, which halves their power relative to the 1.7B ladder. The free-form control uses a 4,096-token budget; the 16,384-token re-run shows that a larger budget recovers few of the 213 truncated generations. The Llama-3.1-8B typing habit was diagnosed from the score files and by comparison with the published prompting-mode number, not by re-running the weights on a second server. The overthinking penalty is one fixed λ rather than the authors’ tuned value. Multi-turn is underpowered at $n = 200$ and a 17% base rate, and was not run on Llama. A repeat control on the

Llama models, a third family, and the multi-turn category on Llama are the next additions.

6 Conclusions

We expected a function signature, having no slack, to be the first thing quantization breaks. On the deployed format, scored deterministically, it was the output that held up best. For Qwen3-1.7B the tool-calling floor and the free-form floor sit at the same rung, Q4_K_M, but the call loses 4.75 points at Q3_K_M where the mathematics loses 39.50, and the difference is length: three-bit weights make the chain of thought grow past the token budget, and a call is complete long before that happens. On Qwen3-8B and Qwen3-14B there is no tool-calling cliff at any rung. On the Llama family the strict score moves in both directions because compression changes how the models type their arguments; scored on the parsed value, four bits costs neither Llama model detectable accuracy and three bits costs Llama-3.1-8B 7.00 points. A fair extractor changes nothing, and a penalty designed for reasoning traces finds little to act on in a call. The bit-width an agent can tolerate has to be measured on the model being shipped, and the damage quantization does to long reasoning does not carry over to the tool call.

Additional Information and Declarations

Competing Interests. The author is employed full time as a software engineer at Amazon (2022 to present). Amazon had no involvement in this work, which was done independently, on the author’s own hardware and time; none of the models, quantization tools, benchmarks or inference servers evaluated is an Amazon product. The author has no financial or other interest in any of them, and none of their developers was involved in the study.

Author Contributions. Akshay Kumar conceived and designed the experiments, performed the experiments, analyzed the data, prepared figures and tables, authored and reviewed drafts of the article, and approved the final draft.

Funding. The author received no funding for this work. All computation ran on the author’s own hardware.

Data Availability. The code, run scripts, every raw BFCL result and score file, every GSM8K generation, the server logs, the pre-registration and the dated lab notebook are available at <https://github.com/akshayaggarwal99/quant-tool-calling> under the MIT licence, at the release tag v1.2-peerj, which is the version this paper describes. A DOI-bearing archive of that tag will be deposited at Zenodo on acceptance. The GGUF weight files are not included; the scripts rebuild them from the published FP16 checkpoints.

References

Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do

NOT Think That Much for $2+3=?$ On the Overthinking of o1-Like LLMs. *arXiv preprint arXiv:2412.21187*, 2024.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.

Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit Inference Scaling Laws. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems 35*, 2022.

Ercin Dinger and Zeynep Hilal Kilimci. Real-time and offline large language models on edge devices: a systematic review. *PeerJ Computer Science*, 12:e3769, 2026. doi: 10.7717/peerj-cs.3769.

Ravitez Dondeti. ToolGuard: Red-Teaming Small Language Model Tool Calling on Consumer Hardware. *Transactions on Machine Learning Research*, 2026.

Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations*, 2023.

Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023.

Gorilla Team, UC Berkeley. Berkeley Function Calling Leaderboard V4: leaderboard data (data_non_live.csv). <https://gorilla.cs.berkeley.edu/leaderboard.html>, 2026. Data file last modified 13 April 2026; retrieved 29 September 2026.

Konstantin Hebenstreit, Robert Praas, Louis P. Kiesewetter, and Matthias Samwald. A comparison of chain-of-thought reasoning strategies across datasets and models. *PeerJ Computer Science*, 10:e1999, 2024. doi: 10.7717/peerj-cs.1999.

Renren Jin, Jiangcun Du, Wuwei Huang, Wei Liu, Jian Luan, Bin Wang, and Deyi Xiong. A Comprehensive Evaluation of Quantization Strategies for Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Tanishq Kumar, Zachary Ankner, Benjamin F. Spector, Blake Bordelon, Niklas Muennighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. Scaling Laws for Precision. In *International Conference on Learning Representations*, 2025.

Uygar Kurt. Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct. *arXiv preprint arXiv:2601.14277*, 2026.

Eldar Kurtic, Alexandre Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. “Give Me BF16 or Give Me Death”? Accuracy-Performance Trade-Offs in LLM Quantization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26872–26886, 2025. doi: 10.18653/v1/2025.acl-long.1304.

Jemin Lee, Sihyeong Park, Jinse Kwon, Jihun Oh, and Yongin Kwon. Exploring the Trade-Offs: Quantization Methods, Task Difficulty, and Model Size in Large Language Models From Edge

to Giant. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25)*, pages 8113–8121, 2025. doi: 10.24963/ijcai.2025/902.

Shiyao Li, Xuefei Ning, Luning Wang, Tengxuan Liu, Xiangsheng Shi, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. Evaluating Quantized Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Machine Learning and Systems (MLSys)*, 2024.

Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization Hurts Reasoning? An Empirical Study on Quantized Reasoning Models. In *Conference on Language Modeling (COLM)*, 2025.

Llama Team, AI @ Meta. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.

Sanae Lotfi, Polina Kirichenko, Steven Li, and Zechun Liu. Quantized Reasoning Models Think They Need to Think Longer, but They Do Not. *arXiv preprint arXiv:2606.00206*, 2026.

Jinendra Malekar and Ramtin Zand. Amdahl’s law for LLMs: A throughput-centric analysis of extreme LLM quantization. *Transactions on Machine Learning Research*, 2025.

Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. How Does Quantization Affect Multilingual LLMs? In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

Quinn McNemar. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2):153–157, 1947.

Anmol Mekala, Anirudh Atmakuru, Yixiao Song, Marzena Karpinska, and Mohit Iyyer. Does Quantization Affect Models’ Performance on Long-Context Tasks? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9422–9470, 2025. doi: 10.18653/v1/2025.emnlp-main.479.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems* 37, 2024.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 48371–48392, 2025.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*, 2024.

Qwen Team. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.

Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, Hanjie Chen, and Xia Hu. Stop Overthinking: A

Survey on Efficient Reasoning for Large Language Models. *Transactions on Machine Learning Research*, 2025.

Hazal Turkmen. Generative small language models in clinical NLP: applications, adaptation, and evaluation. *PeerJ Computer Science*, 12:e4000, 2026. doi: 10.7717/peerj-cs.4000.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35*, 2022.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations*, 2025.